



FUNÇÕES FRIEND, CONSTRUTOR DE CÓPIA E SOBRECARGA DE OPERADORES PARTE 1

Prof. Dr. Valter Vieira de Camargo

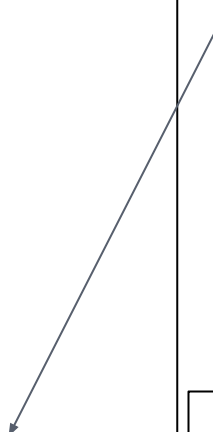
POO - 2026/1

FUNÇÕES FRIEND

- É uma função que não pertence à classe, mas recebe permissão para acessar seus membros privados e protegidos
- Ela, no fundo, quebra o encapsulamento de forma controlada
- É como se você falasse assim para uma classe: *“Olha, existe uma função chamada $f()$ em algum lugar que ela poderá acessar diretamente seus membros privados”*.
- Exemplo →

FUNÇÕES FRIEND

```
class Pessoa {  
private:  
    string nome;  
  
public:  
    Pessoa(string n) {  
        nome = n;  
    }  
  
    friend void mostrarNome(Pessoa p);  
};
```



É como se ela fosse uma função membro da classe (um método da classe)

- Dentro da classe é necessário informar qual função é amiga dela (*colocando a assinatura*). Mesmo que vc não saiba onde essa função foi declarada.
- A função amiga, não importa onde ela esteja, conseguirá acessar membros privados e protegidos diretamente

```
// Função comum (não membro)  
void mostrarNome(Pessoa p) {  
    // Pode acessar membro private  
    cout << p.nome << endl;  
}
```

```
int main() {  
    Pessoa p("Maria");  
    mostrarNome(p);  
    return 0;  
}
```



CONSTRUTOR DE CÓPIA

SOBRECARGA DE MÉTODOS / FUNÇÕES

- Métodos com mesmo nome mas **assinaturas diferentes**: Tipos, ordem ou número de parâmetros diferentes
 - Métodos com a mesma assinatura não podem ter tipos de retorno diferentes
 - Já usamos sobrecarga de metodos em Construtores

Exemplos:

Tipo_retorno **metodoX**(*tipo1* y);

Tipo_retorno **metodoX**(*tipo1* y, *tipo2* z);

Tipo_retorno **metodoX**(*tipo1* y, *tipo3* w);

Tipo_retorno **metodoX**(*tipo2* y, *tipo1* z);

CONSTRUTOR DE CÓPIA

Usado quando se deseja criar um **novo** objeto que seja **uma cópia de outro**, isto é, um **outro objeto** mas com os mesmos dados do primeiro

```
#include <iostream>
using namespace std;
```

```
class Pessoa {
public:
    string nome;
```

```
    Pessoa(string n) : nome(n) {}
```

```
    // Construtor de cópia
```

```
    Pessoa(const Pessoa& outra) {
        cout << "Chamando o construtor de cópia\n";
        nome = outra.nome;
    }
};
```

```
int main() {
    Pessoa p1("Maria");
    Pessoa p2 = p1; // Aqui o construtor de cópia é chamado

    cout << p2.nome << endl;
}
```

Recebe uma referência para um outro objeto do mesmo tipo

Obs. Tem que ser por referência, se não gera looping infinito

Veja que um outro objeto é criado sem ter que chamar o construtor diretamente, como foi feito com p1.

Mas poderia ser também →
Pessoa p2(p1);

CONSTRUTOR DE CÓPIA

Se você não declarar um construtor de cópia, o compilador gera automaticamente um que **copia membro a membro** (chamado de ***shallow copy***)

```
#include <iostream>
using namespace std;
```

```
class Pessoa {
public:
    string nome;
```

```
    Pessoa(string n) : nome(n) {}
```

```
    // Construtor de cópia
```

```
    Pessoa(const Pessoa& outra) {
        cout << "Chamando o construtor de cópia\n";
        nome = outra.nome;
    }
```

```
};
```

```
int main() {
```

```
    Pessoa p1("Maria");
```

```
    Pessoa p2 = p1; // Aqui o construtor de cópia é chamado
```

```
    cout << p2.nome << endl;
```

```
}
```

Recebe uma referência para um outro objeto do mesmo tipo

Obs. Tem que ser por referência, se não gera looping infinito

Veja que um outro objeto é criado sem ter que chamar o construtor diretamente, como foi feito com p1.

Mas poderia ser também →
Pessoa p2(p1);

CONSTRUTOR DE CÓPIA

Onde mais ele é chamado ? Ele é chamado sempre que uma cópia for necessária...

```
#include <iostream>
using namespace std;

class Pessoa {
public:
    string nome;

    Pessoa(string n) : nome(n) {}

    Pessoa(const Pessoa& outra) {
        cout << "Construtor de cópia chamado!\n";
        nome = outra.nome;
    }
};

void imprimePessoa(Pessoa p) {
    cout << "Nome: " << p.nome << endl;
}

int main() {
    Pessoa p1("João");
    imprimePessoa(p1); // ← Aqui o construtor de cópia também é chamado
}
```

Declaração de função **global (fora da classe)** que recebe por parâmetro um objeto do tipo Pessoa. Veja que recebe um objeto e não uma referência. O objeto p é uma cópia do p1 que foi passado por parâmetro

É a mesma coisa que Pessoa
 $p = p1$

CONSTRUTOR DE CÓPIA

Note que é meio perigoso não declarar o construtor de cópia e confiar no “shallow copy”

O compilador copia cada membro membro a membro

É perigoso quando você usa ponteiros

```
class Vetor {  
public:  
    int* dados;  
  
    Vetor(int tamanho) {  
        dados = new int[tamanho];  
    }  
  
    ~Vetor() {                //destrutor  
        delete[] dados;  
    }  
};
```

- Não declarou o construtor de cópia
- dados é um array de inteiros

Aqui, **v1** e **v2** apontam **para a mesma área de memória**. Quando um deles for destruído (saindo de escopo, por exemplo), ele vai dar **delete[]** em **dados**, e o outro vai ficar com um ponteiro inválido (erro de memória).

```
Vetor v1(10);  
Vetor v2 = v1; // ← Copia superficial (mesmo ponteiro!)  
                A intenção seria ter dois vetores diferentes com  
                dois arrays diferentes dentro dele.
```

O compilador irá fazer → **v2.dados = v1.dados;**
fazendo os dois objetos apontarem para a mesma localização de memória (o mesmo array)

CONSTRUTOR DE CÓPIA

Note que é meio perigoso não declarar o construtor de cópia e usar o “shallow copy”

O compilador copia cada membro membro a membro

É perigoso quando você usa ponteiros

```
class Vetor {  
public:  
    int* dados;  
  
    Vetor(int tamanho) {  
        dados = new int[tamanho];  
    }  
  
    ~Vetor() {                //destrutor  
        delete[] dados;  
    }  
};
```

- Não declarou o construtor de cópia
- dados é um array de ponteiros

Aqui, **v1** e **v2** apontam **para a mesma área de memória**. Quando um deles for destruído (saindo de escopo, por exemplo), ele vai dar **delete[]** em **dados**, e o outro vai ficar com um ponteiro inválido (erro de memória).

```
Vetor v1(10);  
Vetor v2 = v1; // ← Copia superficial (mesmo ponteiro!)  
                A intenção seria ter dois vetores diferentes com  
                dois arrays diferentes dentro dele.
```

Quando v1 e v2 saírem do escopo, o destrutor será chamado 2x. O segundo tentará liberar a mesma memória já liberada.

CONSTRUTOR DE CÓPIA

Solução é declarar seu próprio construtor de cópia (**Deep Copy**)

```
Vetor(const Vetor& outro) {  
    // aloca nova memória  
    dados = new int[10];  
  
    for (int i = 0; i < 10; i++) {  
  
        dados[i] = outro.dados[i]; // cópia manual  
  
    }  
}
```

```
Vetor v1(10);  
Vetor v2 = v1;
```

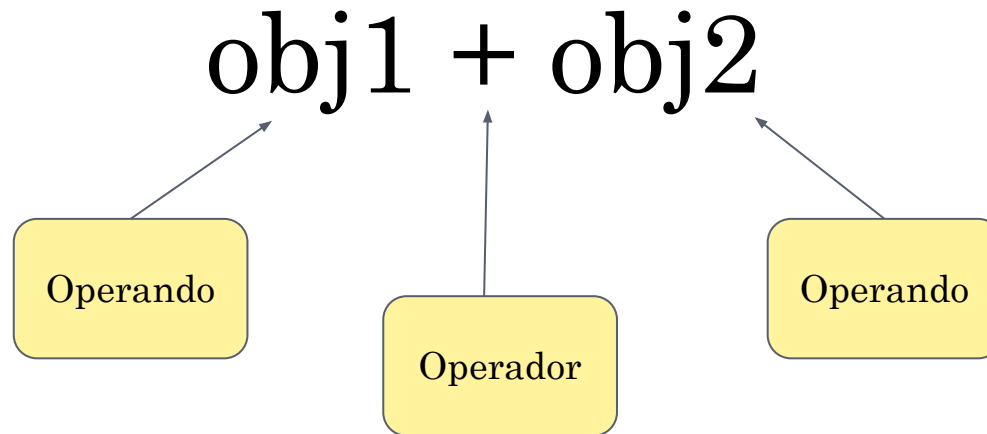
ClasseX novoObj(velhoObj);

Outra forma de chamar o
construtor de cópia !
sem usar o “igual/atribuição”



SOBRECARGA DE OPERADORES

SOBRECARGA DE OPERADORES



SOBRECARGA DE OPERADORES

- ❑ O que é Sobrecarga de Operadores ?
- ❑ Significa usar operadores (+, *, -, /, @, etc) para coisas que eles não foram criados
- ❑ Normalmente é para fazer operações entre classes/objetos
- ❑ Exemplo

Considere a criação de um método soma:

● `objeto3 = objeto1.soma(objeto2);`

? Passando o objeto 2 para o método soma do objeto1

● `objeto3 = objeto1 + objeto2;`

● `= objeto1.operator+(objeto2)`

Muito mais claro!

SOBRECARGA DE OPERADORES

Operadores que podem ser sobrecarregados							
+	-	*	/	%	^	&	
~	!	=	<	>	+=	-=	*=
/=	%=	^=	&=	=	<<	>>	>>=
<<=	==	!=	<=	>=	&&		++
--	->*	,	->	[]	()	new	delete
new[]	delete[]						

Operadores que não podem ser sobrecarregados				
.	.*	::	?:	sizeof

SOBRECARGA DE OPERADORES

- Uso de operadores c/ objetos
 - Utilização de operadores da linguagem para manipular objetos
 - Clareza
 - Operadores se tornam sensíveis ao contexto
- Exemplo
 - +
 - Normalmente usado só para operações aritméticas (integers, floats, etc.)
 - Se você sobrecarregar o +, poderá fazer:
 - Matriz A, B, C;
 - **C = A+B;**

RESTRIÇÕES

- Não é possível alterar:
 - Como os operadores trabalham com os tipos **pré-definidos**
 - Soma de inteiros deve ser sempre soma de inteiros
 - Precedência de operadores (ordem de avaliação da expressão)
 - Use parênteses p/ forçar mudança na ordem
 - Associatividade (esq-p/-direita ou direita-p/-esq)
 - Número de operandos
 - **&** é unário, ou seja, tem somente um operando
 - **+** é binário dois operandos
- **Não se pode criar novos operadores**
- Operadores devem ser sobrecarregados explicitamente
 - Sobrecarga de **+** não sobrecarrega **+=**

FUNDAMENTOS DE SOBRECARGA DE OPERADORES

□ Sobrecarga de operadores: Como fazer?

- Crie uma função para a classe
- Nomeie a função com o nome **operator** seguida pelo símbolo dele.
 - **Operator+** para o operador de adição

● `objeto3 = objeto1 + objeto2;`

OU...

● `objeto1.operator+(objeto2)`

FUNÇÕES OPERADORES COMO FUNÇÕES MEMBRO OU FRIEND ?

□ Funções Operadores

- Como funções Membro/ métodos
(**Preferencialmente**)

- Use **this** p/ obter argumentos implicitamente
- Obtém operando à **esquerda** para operador binário (como *****)
- Objeto mais à esquerda deve ser da mesma classe que o operador sendo sobrecarregado

- Como funções não membro/globais

- Necessitam parâmetros para ambos os operandos
- Podem ter objeto de classe diferente da do operador
- Tem que ser **friend** para acessar dados **private** ou **protected**

EXEMPLO

Imagine que você queira somar dois vetores 3D.

Você quer fazer isso:

```
Vetor3D a(1, 2, 3);
```

```
Vetor3D b(4, 5, 6);
```

```
Vetor3D c = a + b; // ← Isso só funciona se  
você sobrecarregar o operador +
```

EXEMPLO

```
class Vetor3D {  
public:  
    float x, y, z;
```

```
Vetor3D(float x, float y, float z)  
    : x(x), y(y), z(z) {}
```

```
// Sobrecarga do operador +
```

```
Vetor3D operator+(const Vetor3D& outro) const {
```

```
    return Vetor3D(x + outro.x, y + outro.y, z + outro.z);
```

```
}
```

```
};
```

Quando você sobrecarrega um operador como método-membro, o operando à esquerda é o **objeto que chama o método (`this`)**.
O operando à direita é o argumento passado para o operador.

```
Vetor3D a(1, 2, 3), b(4, 5, 6);
```

```
Vetor3D c = a + b; ← Agora sim ! //chamada à função operator+
```

EXEMPLO

TAMBÉM PODE-SE CRIAR UMA FUNÇÃO OPERADOR NÃO MEMBRO (FORA DE CLASSES).

AGORA COM DOIS PARÂMETROS/ARGUMENTOS

```
Vetor3D operator+(const Vetor3D& a, const Vetor3D& b) {  
    return Vetor3D(a.x + b.x, a.y + b.y, a.z + b.z);  
}
```

```
class Vetor3D {  
    ....
```

Lembre-se que dentro da classe deve-se dizer que ela é uma função amiga !

```
// Friend para permitir acesso aos privados
```

```
friend Vetor3D operator+(const Vetor3D& a, const Vetor3D& b
```

```
    ...
```

```
};
```

```
Vetor3D a(1, 2, 3), b(4, 5, 6);
```

```
Vetor3D c = a + b; Agora sim !
```

EXEMPLO

SOBBRECARGA DE OPERADOR UNÁRIO

- SUPONHA QUE VOCÊ QUEIRA SOBRECARREGAR O OPERADOR `++` E `--` PARA DESLOCAR PONTOS EM UM PLANO CARTESIANO
- POR EXEMPLO

Ponto `p(3, 4);`

`++p;` // Depois disso `p` é `(4, 5)`

Detalhe antes de continuar

DIFERENÇA PRÉ-FIXADO E PÓS-FIXADO

Qual a diferença entre ++X e X++ ?

```
int x = 5;  
int y = ++x;  
resultado ?
```

```
int x = 5;  
int y = x++;  
resultado ?
```

```
return this;
```

```
}
```


Detalhe antes de continuar

Qual a diferença entre ++X e X++ ?

```
int x = 5;  
int y = ++x;  
resultado ? x = 6 e y = 6
```

```
int x = 5;  
int y = x++;  
resultado ? x = 6 e y = 5
```

```
return this;
```

```
}
```

SOBRECARGA DE OPERADOR UNÁRIO (PRÉ-FIXADA)

```
class Ponto {  
private: int x, y;  
public:  
    Ponto(int x = 0, int y = 0)  
        : x(x), y(y) {}  
  
    Ponto& operator++() {  
        ++x;  
        ++y;  
        return *this;  
    }  
  
    Ponto& operator--() {  
        --x;  
        --y;  
        return *this;  
    }  
};
```

```
int main() {  
    Ponto p(2, 3);  
  
    cout << "Valor original: ";  
    p.exibir();  
  
    ++p; // Chama operator++()  
    cout << "Após ++p: ";  
    p.exibir();  
  
    --p; // Chama operator--()  
    cout << "Após --p: ";  
    p.exibir();  
}
```

SOBBRECARGA DE OPERADOR UNÁRIO (PRÉ-FIXADA)

```
class Ponto {  
private: int x, y;  
public:  
    Ponto(int x = 0, int y = 0)  
        : x(x), y(y) {}
```

```
    Ponto& operator++() {  
        ++x;  
        ++y;  
        return *this;  
    }
```

```
    Ponto& operator--() {  
        --x;  
        --y;  
        return *this;  
    }
```

- *this permite fazer encadeamento. Por exemplo: ++++p, ++p++.exibir()
- Ponto&: a função retorna uma referência para o próprio objeto (*this),

EXERCÍCIOS

- 1) Implemente uma classe `StringSegura` que armazena uma string (como um `char*` ou `std::string`). Sobrecarregue o operador `[]` para acessar os caracteres da string com verificação de limites.
 - Se o índice estiver fora dos limites, retorne `'?'` ou gere uma mensagem de erro.
 - Permita acesso somente leitura (versão `const` do operador).

Exemplo de uso:

```
StringSegura s("teste");
```

```
cout << s[1];    // e
cout << s[10];   // ? ou erro
```

```
#include <iostream>
#include <string>
using namespace std;

class StringSegura {
private:
    string texto;

public:
    StringSegura(const string& s) : texto(s) {}
```

Dica:

- A classe `String` possui um método chamado `size()` que retorna o tamanho da `String`

EXERCÍCIOS

Implemente uma classe `Vetor3D` com três coordenadas `x`, `y`, `z`. Sobrecarregue o operador `*` para realizar o **produto escalar (dot product)** entre dois vetores.

- Dica: o produto escalar de $A(x_1, y_1, z_1)$ e $B(x_2, y_2, z_2)$ é $x_1*x_2 + y_1*y_2 + z_1*z_2$.

Exemplo de uso:

```
Vetor3D a(1, 2, 3);  
Vetor3D b(4, 5, 6);
```

```
int resultado = a * b; // 1*4 + 2*5 + 3*6 = 32
```

EXERCÍCIOS

Exercício 3 — Operador `==` (comparação personalizada)

Enunciado:

Crie uma classe `Data` com atributos `dia`, `mes`, `ano`. Sobrecarregue o operador `==` para comparar duas datas e retornar `true` se forem exatamente iguais.

Depois, adicione também o operador `!=` com base no `==`.

Exemplo de uso:

```
Data d1(2, 6, 2025);  
Data d2(2, 6, 2025);  
Data d3(3, 6, 2025);  
  
cout << (d1 == d2);    // true  
cout << (d1 != d3);    // true
```